Part C: Results and Findings

Held-out performance, paired statistical comparison and recommendation

Author	Vusumuzi Nkosi
Student number	26303296
Module	CAP182 Capstone Project
Document status	Word review report — no final PDF created

Report contents

- Model 1 performance: Logistic Regression
- Model 2 performance: XGBoost
- Model comparison, McNemar test and operating trade-off

Model 1 Performance - Logistic Regression (CAP182 SS2 - Part C)

Related scripts	scripts/train_model1_logreg.py and scripts/evaluate.py

How to run

```
python scripts/train_model1_logreg.py
```

The script trains Model 1 and evaluates it once on the fixed 8,238-row held-out test set at a probability threshold of 0.50.

Results

Metric	Value
Accuracy	0.8339
Balanced accuracy	0.7536
Precision	0.3663
Recall (sensitivity)	0.6498
Specificity	0.8573
F1	0.4685
ROC-AUC	0.8020
PR-AUC (average precision)	0.4591

Confusion matrix (tn, fp, fn, tp): **6,267, 1,043, 325, 603**

Interpretation

Accuracy is not the most prominent metric in this context since only 928 out of the total of 8,238 test observations (11.3%) are positive. A simple no classification would achieve higher accuracy while detecting no subscribers at all. In turn, balanced accuracy, recall, precision, F1 and PR-AUC will give more insight into model performance.

With a conventional 0.50 threshold, Logistic Regression can detect 603 of 928 positive subscribers achieving recall equal to 0.650. This is quite a decent screening capacity; however, the presence of 1,043 false positives brings down precision to 0.366 – only about 37 out of 100 customers detected will be actually subscribing ones. Consequently, a large number of contacts would be initiated with people not being subscribers, which implies additional costs.

The PR-AUC of 0.459 exceeds the positive base rate of 0.113 by the factor of approximately 4.1. Therefore, there is an evident ranking capacity above the level of no-skill classifier. The ROC-AUC is 0.802, demonstrating good class separation capacity, and balanced accuracy is 0.754, proving that the model works sufficiently well.

Model 2 Performance - XGBoost (CAP182 SS2 - Part C)

Related scripts	scripts/train_model2_xgboost.py and scripts/evaluate.py

How to run

```
python scripts/train_model2_xgboost.py.
```

This script runs Model 2 on the same exact 8,238 row holdout set and threshold of 0.50 as used for Model 1.

Results

Metric	Value
Accuracy	0.8496
Balanced accuracy	0.7614
Precision	0.3972
Recall (sensitivity)	0.6476
Specificity	0.8752
F1	0.4924
ROC-AUC	0.8090
PR-AUC (average precision)	0.4868

Confusion matrix (tn, fp, fn, tp): **6,398, 912, 327, 601**

Interpretation

Model 2 correctly identifies 601 out of 928 actual subscribers, making a recall score of 0.648. This is approximately equal coverage of Logistic Regression, although Model 2 creates 912 false positives compared to Logistic Regression's 1,043. Thus, precision becomes 0.397 – meaning that approximately 40 out of every 100 customers marked as potentially subscribing in fact do. This is helpful for a resource-limited campaign team since it lowers wasted contacts without substantially lowering the number of potential subscribers contacted.

It also increases balanced accuracy, specificity, F1, ROC-AUC and PR-AUC. Its PR-AUC of 0.487 is more than four times larger than the 0.113 positive class base rate, making it particularly beneficial for this class-imbalanced dataset. While there is a difference of two true positives from Model 1, there is a difference of 131 false negatives, so the tradeoff between these depends on the cost of missed activators versus unneeded contacts.

The improvement makes sense as the interaction and non-linearity models that boosted tree algorithms can make are outside the reach of the linear log-odds model of Logistic Regression. However, this comes with a decrease in transparency of the individual prediction. A generated feature importance report partially solves this issue, although other measures are needed in production.

Comparison of Model 1 and Model 2 (CAP182 SS2 - Part C)

Related scripts	scripts/compare_models.py and scripts/explain_models.py

How to run

```
python scripts/train_model1_logreg.py
python scripts/train_model2_xgboost.py
python scripts/explain_models.py
python scripts/compare_models.py
```

The two algorithms use identical training/testing stratification method, seed, engineered features, and withheld rows, so their outputs can be directly compared to one another.

Metrics side by side

Metric	Model 1: Logistic Regression	Model 2: XGBoost	Difference (M2 - M1)
Accuracy	0.8339	0.8496	+0.0157
Balanced accuracy	0.7536	0.7614	+0.0079
Precision	0.3663	0.3972	+0.0309
Recall	0.6498	0.6476	-0.0022
Specificity	0.8573	0.8752	+0.0179
F1	0.4685	0.4924	+0.0239
ROC-AUC	0.8020	0.8090	+0.0070
PR-AUC	0.4591	0.4868	+0.0277

The XGBoost algorithm at threshold of 0.50 eliminates 131 false positives (912 versus 1,043), at the cost of two true positives (601 versus 603). Thus, it achieves the practical coverage while reducing the efficiency of the positive classification. Higher PR-AUC is especially significant since the positive class accounts for just 11.3% of the test set.

Paired statistical comparison

McNemar's test considers only cases on which the models disagree:

- Model 1 correct and Model 2 wrong (b): 214
- Model 1 wrong and Model 2 correct (c): 343
- Continuity-corrected chi-square: 29.415
- Chi-square p-value: 5.84 x 10^-8
- Exact binomial p-value: 5.12 x 10^-8

The p-value is well below 0.05 and c is greater than b; thus, XGBoost is right more often on the discordant pairs. Significance does not necessarily equal business value, but in this case the two seem consistent with each other.

ROC and precision-recall curves

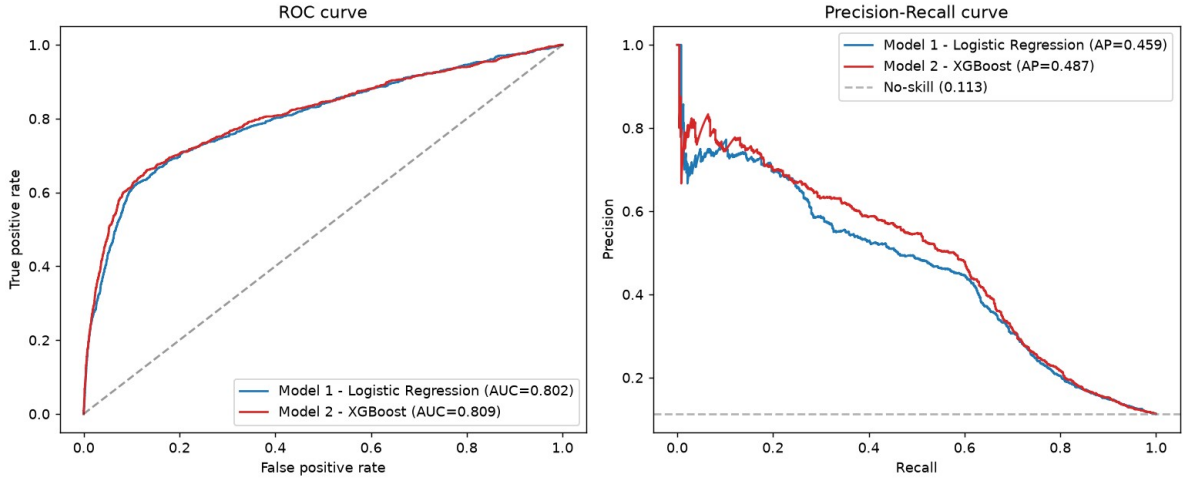


Figure: Held-out ROC and precision-recall curves

The curves illustrate a small ROC gain and a clearer precision-recall benefit for XGBoost. As ROC curves are overly optimistic for imbalanced datasets, the precision-recall chart and its no-skill baseline are more important for the decision.

Trade-off and verdict

Logistic Regression retains its value for cases where simplicity and interpretability of coefficients and governance outweigh other factors. XGBoost is not as easily interpretable as Logistic Regression but performs better in terms of accuracy, balanced accuracy, precision, specificity, F1, ROC-AUC, and PR-AUC with no change to the recall rate. Paired tests show that the difference is not likely to be due to sampling errors.

XGBoost (Model 2) is suggested for the proof-of-concept with public data. Prior to any production implementation, a proper threshold should be selected based on outreach capacities and costs of mistakes, probabilities need to be calibrated, and its performance needs to be checked on separate STADIOEquities data.